

YOUTUBE SHORTS

RECOMMENDATION SYSTEM

Complete Production Guide

VOLUME 2

Model Development & Training

2025-2026 Edition

Two-Tower Models | Deep Cross Networks | Transformers | Multi-Task Learning

Table of Contents

Chapter 7: Candidate Generation Models 3

Chapter 8: Ranking Models Deep Dive 18

Chapter 9: Multi-Task & Multi-Objective Learning 32

Chapter 10: Cold Start Problem Solutions 45

Chapter 11: Training Infrastructure & Optimization 58

Chapter 12: Model Evaluation & Offline Metrics 70

Appendix C: Loss Functions Reference 82

Appendix D: Hyperparameter Tuning Guide 86

Chapter 7: Candidate Generation Models

7.1 The Role of Candidate Generation

Candidate generation is the first and most critical stage of a recommendation pipeline. Its purpose is to efficiently narrow down millions of potential items to a manageable set of hundreds or thousands that are most likely to be relevant to the user. This stage must balance two competing objectives: high recall (not missing relevant items) and computational efficiency (responding in milliseconds).

The fundamental insight behind modern candidate generation is that we can learn dense vector representations (embeddings) for both users and items such that the relevance of an item to a user can be approximated by the similarity of their embeddings.

7.2 Two-Tower Architecture Fundamentals

The two-tower architecture consists of two separate neural networks: a query tower (user tower) that encodes user features, and a candidate tower (item tower) that encodes item features. Both towers produce embeddings of the same dimension, typically 64 to 256.

The key design principle is that the towers are independent at inference time. Item embeddings can be pre-computed and indexed offline, while only the user embedding needs to be computed in real-time.

7.2.1 Mathematical Foundation

Given a user u with features x_u and an item i with features x_i , the two towers learn embedding functions:

User embedding: $e_u = f_{\text{user}}(x_u; \theta_{\text{user}})$

Item embedding: $e_i = f_{\text{item}}(x_i; \theta_{\text{item}})$

The relevance score is computed as: $\text{score}(u, i) = e_u \cdot e_i$

TensorFlow Recommenders Implementation

```
import tensorflow as tf
```

```
import tensorflow_recommenders as tfrs
```

```

class UserTower(tf.keras.Model):

    def __init__(self, user_ids, embedding_dim=128, hidden_dims=[256, 128]):

        super().__init__()

        self.user_lookup = tf.keras.layers.StringLookup(vocabulary=user_ids)

        self.user_embedding = tf.keras.layers.Embedding(len(user_ids)+1, 64)

        self.country_emb = tf.keras.layers.Embedding(250, 16)

        self.device_emb = tf.keras.layers.Embedding(10, 8)

        self.dense_layers = []

        for dim in hidden_dims:

            self.dense_layers.extend([

                tf.keras.layers.Dense(dim, activation='relu'),

                tf.keras.layers.BatchNormalization(),

                tf.keras.layers.Dropout(0.2)

            ])

        self.output = tf.keras.layers.Dense(embedding_dim)

        self.normalize = lambda x: tf.nn.l2_normalize(x, axis=1)

    def call(self, features):

        user_emb = self.user_embedding(self.user_lookup(features['user_id']))

        country = self.country_emb(features['country_id'])

        device = self.device_emb(features['device_type'])

        x = tf.concat([user_emb, country, device, features['continuous']],
axis=1)

```

```

    for layer in self.dense_layers:

        x = layer(x)

    return self.normalize(self.output(x))

```

Item Tower Implementation

```

class ItemTower(tf.keras.Model):

    def __init__(self, video_ids, creator_ids, categories,
embedding_dim=128):

        super().__init__()

        self.video_lookup =
tf.keras.layers.StringLookup(vocabulary=video_ids)

        self.video_embedding = tf.keras.layers.Embedding(len(video_ids)+1,
64)

        self.creator_lookup =
tf.keras.layers.StringLookup(vocabulary=creator_ids)

        self.creator_embedding = tf.keras.layers.Embedding(len(creator_ids)
+1, 32)

        self.category_lookup =
tf.keras.layers.StringLookup(vocabulary=categories)

        self.category_embedding = tf.keras.layers.Embedding(len(categories)
+1, 16)

        self.content_proj = tf.keras.layers.Dense(64, activation='relu')

        self.dense_layers = [

            tf.keras.layers.Dense(256, activation='relu'),

            tf.keras.layers.BatchNormalization(),

            tf.keras.layers.Dropout(0.2),

            tf.keras.layers.Dense(128, activation='relu'),

```

```

        tf.keras.layers.BatchNormalization(),

        tf.keras.layers.Dropout(0.2)

    ]

    self.output = tf.keras.layers.Dense(embedding_dim)

    self.normalize = lambda x: tf.nn.l2_normalize(x, axis=1)

    def call(self, features):

        video = self.video_embedding(self.video_lookup(features['video_id']))

        creator =
self.creator_embedding(self.creator_lookup(features['creator_id']))

        category =
self.category_embedding(self.category_lookup(features['category']))

        content = self.content_proj(features['content_embedding'])

        x = tf.concat([video, creator, category, content,
features['engagement']], axis=1)

        for layer in self.dense_layers:

            x = layer(x)

        return self.normalize(self.output(x))

```

7.3 Training the Two-Tower Model

Complete Retrieval Model with Training Task

```

class TwoTowerRetrievalModel(tf.nn.Model):

    def __init__(self, user_tower, item_tower, candidates, temperature=0.05):

        super().__init__()

        self.user_tower = user_tower

```

```

        self.item_tower = item_tower

        self.task = tfrs.tasks.Retrieval(

            metrics=tfrs.metrics.FactorizedTopK(

                candidates=candidates.batch(128).map(lambda x:
self.item_tower(x)),

                ks=[10, 50, 100]

            ),

            temperature=temperature,

            remove_accidental_hits=True

        )

def compute_loss(self, features, training=False):

    user_emb = self.user_tower(features['user_features'])

    item_emb = self.item_tower(features['item_features'])

    return self.task(user_emb, item_emb, compute_metrics=not training)

# Training with best practices

def train_model(model, train_ds, val_ds, epochs=10, batch_size=8192,
lr=0.001):

    tf.keras.mixed_precision.set_global_policy('mixed_float16') # Faster
training

    lr_schedule = tf.keras.optimizers.schedules.CosineDecay(

        initial_learning_rate=lr, decay_steps=epochs * len(train_ds) //
batch_size, alpha=0.1

    )

```

```

optimizer = tf.keras.optimizers.Adam(learning_rate=lr_schedule,
clipnorm=1.0)

model.compile(optimizer=optimizer)

callbacks = [

    tf.keras.callbacks.EarlyStopping(

        monitor='val_factorized_top_k/top_100_categorical_accuracy',

        patience=3, mode='max', restore_best_weights=True

    ),

    tf.keras.callbacks.ModelCheckpoint('checkpoints/best.keras',
save_best_only=True)

]

return
model.fit(train_ds.shuffle(100000).batch(batch_size).prefetch(tf.data.AUTOTUNE),

        validation_data=val_ds.batch(batch_size), epochs=epochs,
callbacks=callbacks)

```

7.4 Building the ANN Index

FAISS Index Construction for Fast Retrieval

```

import faiss

import numpy as np

class CandidateIndex:

    def __init__(self, embedding_dim=128, index_type='IVF4096,PQ32',
use_gpu=True):

        self.embedding_dim = embedding_dim

```



```

        self.index_type = index_type

        self.use_gpu = use_gpu

        self.index = None

        self.id_mapping = {}

    def build_index(self, embeddings, video_ids, train_size=100000):

        faiss.normalize_L2(embeddings) # For cosine similarity

        self.index = faiss.index_factory(self.embedding_dim, self.index_type,

                                         faiss.METRIC_INNER_PRODUCT)

        if not self.index.is_trained:

            sample = np.random.choice(len(embeddings), min(train_size,
len(embeddings)), False)

            self.index.train(embeddings[sample])

        self.index.add(embeddings)

        self.id_mapping = {i: vid for i, vid in enumerate(video_ids)}

        if self.use_gpu and faiss.get_num_gpus() > 0:

            res = faiss.StandardGpuResources()

            self.index = faiss.index_cpu_to_gpu(res, 0, self.index)

    def search(self, query_embeddings, k=1000, nprobe=64):

        query_embeddings = query_embeddings.astype('float32')

        faiss.normalize_L2(query_embeddings)

        if hasattr(self.index, 'nprobe'): self.index.nprobe = nprobe

        scores, indices = self.index.search(query_embeddings, k)

```

```
        video_ids = [[self.id_mapping.get(idx) for idx in row] for row in
indices]

    return scores, video_ids
```

Chapter 8: Ranking Models Deep Dive

8.1 The Ranking Stage

The ranking stage receives candidate items from retrieval and produces fine-grained relevance scores. Unlike retrieval which must be fast, ranking can afford more computational complexity since it operates on hundreds rather than millions of items.

8.2 Deep Cross Network (DCN-V2)

DCN-V2 is state-of-the-art for learning explicit feature interactions. It combines a cross network for polynomial feature interactions with a deep network for implicit interactions.

DCN-V2 Implementation

```
class CrossLayer(tf.keras.layers.Layer):

    def __init__(self, projection_dim=None):

        super().__init__()

        self.projection_dim = projection_dim

    def build(self, input_shape):

        dim = input_shape[-1]

        if self.projection_dim:

            self.U = self.add_weight('U', (dim, self.projection_dim))

            self.V = self.add_weight('V', (self.projection_dim, dim))

        else:

            self.W = self.add_weight('W', (dim, dim))

            self.bias = self.add_weight('bias', (dim,), initializer='zeros')

    def call(self, x0, x1):
```

```

    if self.projection_dim:

        cross = tf.matmul(tf.matmul(x1, self.U), self.V)

    else:

        cross = tf.matmul(x1, self.W)

    return x0 * (cross + self.bias) + x1


class DCNRankingModel(tf.keras.Model):

    def __init__(self, input_dim, cross_layers=3, deep_layers=[512, 256,
128]):

        super().__init__()

        self.cross_layers = [CrossLayer(projection_dim=64) for _ in
range(cross_layers)]

        self.deep = []

        for dim in deep_layers:

            self.deep.extend([tf.keras.layers.Dense(dim, activation='relu'),

                             tf.keras.layers.BatchNormalization(),

                             tf.keras.layers.Dropout(0.3)])

        self.combine = tf.keras.layers.Dense(128, activation='relu')

        self.output_layer = tf.keras.layers.Dense(1, activation='sigmoid')


    def call(self, x, training=False):

        x0 = x1 = x

        for layer in self.cross_layers:

            x1 = layer(x0, x1)

        deep_out = x

```

```

    for layer in self.deep:

        deep_out = layer(deep_out, training=training)

    combined = tf.concat([x1, deep_out], axis=1)

    return self.output_layer(self.combine(combined))

```

8.3 Transformer-Based Sequential Ranking

Sequential models capture temporal dynamics in user behavior, modeling the sequence of interactions to understand user intent.

Transformer Ranking Model

```

class TransformerBlock(tf.keras.layers.Layer):

    def __init__(self, embed_dim, num_heads, ff_dim, dropout=0.1):

        super().__init__()

        self.attn = tf.keras.layers.MultiHeadAttention(num_heads,
embed_dim // num_heads)

        self.ffn = tf.keras.Sequential([tf.keras.layers.Dense(ff_dim,
activation='gelu'),

                                         tf.keras.layers.Dense(embed_dim)])

        self.norm1 = tf.keras.layers.LayerNormalization()

        self.norm2 = tf.keras.layers.LayerNormalization()

        self.dropout1 = tf.keras.layers.Dropout(dropout)

        self.dropout2 = tf.keras.layers.Dropout(dropout)

    def call(self, x, mask=None, training=False):

        attn = self.attn(x, x, attention_mask=mask, training=training)

        x = self.norm1(x + self.dropout1(attn, training=training))

```

```

        ffn = self.ffn(x)

        return self.norm2(x + self.dropout2(ffn, training=training))

class SequentialRankingModel(tf.keras.Model):

    def __init__(self, num_items, max_seq=50, embed_dim=128, num_heads=4,
num_layers=2):

        super().__init__()

        self.item_emb = tf.keras.layers.Embedding(num_items+1, embed_dim)

        self.pos_emb = tf.keras.layers.Embedding(max_seq, embed_dim)

        self.blocks = [TransformerBlock(embed_dim, num_heads, 256) for _ in
range(num_layers)]

        self.dense = tf.keras.layers.Dense(256, activation='relu')

        self.output_layer = tf.keras.layers.Dense(1, activation='sigmoid')

    def call(self, inputs, training=False):

        history, candidate = inputs['history'], inputs['candidate']

        seq_len = tf.shape(history)[1]

        x = self.item_emb(history) + self.pos_emb(tf.range(seq_len))

        mask = tf.linalg.band_part(tf.ones((seq_len, seq_len)), -1, 0)

        for block in self.blocks:

            x = block(x, mask=mask, training=training)

        user_repr = x[:, -1, :]

        candidate_emb = tf.squeeze(self.item_emb(candidate), 1)

        combined = tf.concat([user_repr, candidate_emb, user_repr *
candidate_emb], axis=1)

```

```
return self.output_layer(self.dense(combined))
```

8.4 Ranking Loss Functions

Loss Functions Implementation

```
def bpr_loss(pos_scores, neg_scores):

    '''Bayesian Personalized Ranking loss.'''

    return -tf.reduce_mean(tf.math.log_sigmoid(pos_scores - neg_scores))


def pairwise_hinge(pos_scores, neg_scores, margin=1.0):

    '''Hinge loss for pairwise ranking.'''

    return tf.reduce_mean(tf.maximum(0.0, margin - pos_scores + neg_scores))


def approx_ndcg_loss(y_true, y_pred, temperature=1.0):

    '''Differentiable approximation of NDCG.'''

    gains = tf.pow(2.0, y_true) - 1.0

    sorted_gains = tf.sort(gains, direction='DESCENDING', axis=1)

    positions = tf.range(1, tf.shape(sorted_gains)[1] + 1, dtype=tf.float32)

    discounts = tf.math.log1p(positions) / tf.math.log(2.0)

    ideal_dcg = tf.reduce_sum(sorted_gains / discounts, axis=1)

    softmax_scores = tf.nn.softmax(y_pred / temperature, axis=1)

    approx_dcg = tf.reduce_sum(gains * softmax_scores, axis=1)

    return 1.0 - tf.reduce_mean(approx_dcg / (ideal_dcg + 1e-10))
```

Chapter 9: Multi-Task & Multi-Objective Learning

9.1 Why Multi-Task Learning?

Real recommendation systems must optimize for multiple objectives: watch time, engagement (likes, shares), satisfaction, and creator health. These objectives often conflict, requiring careful balancing.

9.2 Multi-Gate Mixture of Experts (MMoE)

MMoE Implementation

```
class MMoE(tf.keras.Model):

    def __init__(self, input_dim, num_experts=6, expert_dim=256,
                  tasks=['watch_time', 'like', 'share', 'skip']):

        super().__init__()

        self.tasks = tasks

        self.experts = [tf.keras.Sequential([

            tf.keras.layers.Dense(expert_dim, activation='relu'),

            tf.keras.layers.BatchNormalization(),

            tf.keras.layers.Dropout(0.2)

        ]) for _ in range(num_experts)]

        self.gates = {task: tf.keras.layers.Dense(num_experts,
            activation='softmax')

                       for task in tasks}

        self.towers = {}

        for task in tasks:

            activation = 'relu' if task == 'watch_time' else 'sigmoid'

            self.towers[task] = [
```



```

        tf.keras.layers.Dense(128, activation='relu'),

        tf.keras.layers.Dropout(0.2),

        tf.keras.layers.Dense(64, activation='relu'),

        tf.keras.layers.Dense(1, activation=activation)

    ]

    def call(self, inputs, training=False):

        expert_outputs = tf.stack([e(inputs, training=training) for e in
self.experts], axis=1)

        outputs = {}

        for task in self.tasks:

            weights = tf.expand_dims(self.gates[task](inputs), -1)

            task_input = tf.reduce_sum(expert_outputs * weights, axis=1)

            x = task_input

            for layer in self.towers[task]:

                x = layer(x, training=training) if hasattr(layer, 'training')
else layer(x)

            outputs[task] = x

        return outputs

```

9.3 Uncertainty-Weighted Loss

Automatic Loss Balancing

```

class UncertaintyWeightedLoss(tf.keras.layers.Layer):

    def __init__(self, num_tasks):

        super().__init__()

```

```

        self.log_vars = self.add_weight('log_vars', (num_tasks,),
initializer='zeros')

```

```

def call(self, losses):

    total = 0.0

    for i, loss in enumerate(losses):

        precision = tf.exp(-self.log_vars[i])

        total += precision * loss + self.log_vars[i]

    return total

```

9.4 Multi-Objective Score Combination

Combining Predictions at Serving Time

```

from dataclasses import dataclass

```

```

@dataclass

```

```

class ObjectiveWeights:

```

```

    watch_time: float = 0.4

    like_prob: float = 0.2

    share_prob: float = 0.15

    skip_prob: float = -0.2

    freshness: float = 0.05

```

```

def compute_ranking_score(predictions, weights, seen_creators=None):

```

```

    score = (weights.watch_time * predictions['watch_time'] +

```

```
weights.like_prob * predictions['like_prob'] +  
weights.share_prob * predictions['share_prob'] +  
weights.skip_prob * predictions['skip_prob'] +  
weights.freshness * predictions['freshness_score'])  
  
if seen_creators:  
    for i, creator in enumerate(predictions['creator_ids']):  
        if creator in seen_creators:  
            score[i] *= 0.9 # Diversity penalty  
  
return score
```

Chapter 10: Cold Start Problem Solutions

10.1 Understanding Cold Start

The cold start problem occurs when we have insufficient data for quality recommendations. Main scenarios include new user cold start (no interaction history), new item cold start (no engagement data), and system cold start (new deployment).

10.2 New User Solutions

Cohort-Based Cold Start Handler

```
from sklearn.cluster import KMeans

import polars as pl

class ColdStartUserHandler:

    def __init__(self, n_cohorts=50):

        self.n_cohorts = n_cohorts

        self.cohort_model = None

        self.cohort_preferences = {}

        self.global_popularity = None

    def fit(self, users, interactions):

        demo = users.select(['country_enc', 'age_enc', 'device_enc',
                              'lang_enc']).to_numpy()

        self.cohort_model = KMeans(n_clusters=self.n_cohorts)

        labels = self.cohort_model.fit_predict(demo)

        users = users.with_columns(pl.lit(labels).alias('cohort_id'))

        cohort_int = interactions.join(users.select(['user_id',
                                                    'cohort_id']), on='user_id')
```

```

    for cid in range(self.n_cohorts):

        cdata = cohort_int.filter(pl.col('cohort_id') == cid)

        scores = cdata.group_by('video_id').agg([

            (pl.col('watch_pct').mean()*0.5 + pl.col('liked').mean()*0.3

+

            pl.col('shared').mean()*0.2).alias('score')

        ]).sort('score', descending=True)

        self.cohort_preferences[cid] = scores.head(1000)
['video_id'].to_list()

        self.global_popularity = interactions.group_by('video_id').agg([

            pl.len().alias('views')]).sort('views',
descending=True).head(1000)['video_id'].to_list()

def get_recommendations(self, user_features, n=500):

    demo = np.array([[user_features['country_enc'],
user_features['age_enc'],

                        user_features['device_enc'],
user_features['lang_enc']]])

    cohort_id = self.cohort_model.predict(demo)[0]

    result = self.cohort_preferences.get(cohort_id, [])[:int(n*0.7)]

    for vid in self.global_popularity:

        if vid not in result and len(result) < n:

            result.append(vid)

    return result

```

10.3 New Item Solutions

Content-Based Item Embedding

```
class ContentEmbedder:

    def __init__(self, text_model='sentence-transformers/all-MiniLM-L6-v2',
output_dim=128):

        from transformers import AutoModel, AutoTokenizer

        self.tokenizer = AutoTokenizer.from_pretrained(text_model)

        self.text_model = AutoModel.from_pretrained(text_model)

        self.image_model = tf.keras.applications.EfficientNetV2S(

            include_top=False, weights='imagenet', pooling='avg')

        self.text_proj = tf.keras.layers.Dense(output_dim)

        self.image_proj = tf.keras.layers.Dense(output_dim)

        self.fusion = tf.keras.layers.Dense(output_dim, activation='relu')


    def embed_video(self, title, description, thumbnail):

        text = f'{title} {description}'

        inputs = self.tokenizer(text, return_tensors='pt', truncation=True,
max_length=128)

        with torch.no_grad():

            text_emb = self.text_model(**inputs).last_hidden_state[:, 0]

            text_emb = self.text_proj(text_emb.numpy())

            image = tf.image.resize(thumbnail, (384, 384))

            image = tf.keras.applications.efficientnet_v2.preprocess_input(image)

            image_emb = self.image_proj(self.image_model(tf.expand_dims(image,
0)))

            combined = tf.concat([text_emb, image_emb], axis=1)

            fused = self.fusion(combined)
```

```
return fused / (tf.norm(fused) + 1e-10)
```

Exploration Boost for New Items

```
def apply_exploration_boost(scores, ages_hours, view_counts,
                             max_boost=0.5, halflife=24, min_views=100):
    freshness = np.exp(-np.log(2) * ages_hours / halflife)
    view_factor = np.clip(1 - view_counts / min_views, 0, 1)
    boost = max_boost * freshness * view_factor
    return scores * (1 + boost)

def thompson_sampling_boost(scores, successes, failures, explore_weight=0.3):
    alpha, beta = successes + 1, failures + 1
    sampled_ctr = np.random.beta(alpha, beta)
    return (1 - explore_weight) * scores + explore_weight * sampled_ctr
```

Chapter 11: Training Infrastructure & Optimization

11.1 Distributed Training

Training recommendation models at scale requires distributed computing. Large embedding tables and massive datasets necessitate sophisticated parallelism strategies.

TorchRec Distributed Training

```
import torch.distributed as dist

from torchrec import EmbeddingBagCollection

from torchrec.distributed import DistributedModelParallel

from torchrec.distributed.planner import EmbeddingShardingPlanner


def setup_distributed(model, embedding_configs, world_size):

    dist.init_process_group(backend='nccl')

    rank = dist.get_rank()

    device = torch.device(f'cuda:{rank}')

    ebc = EmbeddingBagCollection(tables=embedding_configs, device=device)

    planner = EmbeddingShardingPlanner(

        topology=Topology(world_size=world_size, compute_device='cuda'))

    model = DistributedModelParallel(

        module=model, device=device, plan=planner.plan(model,
sharders=[...]))

    return model, device
```

11.2 Hyperparameter Optimization

Ray Tune Hyperparameter Search

```

from ray import tune

from ray.tune.schedulers import ASHAScheduler

from ray.tune.search.optuna import OptunaSearch


def train_with_config(config):

    model = TwoTowerRetrievalModel(

        embedding_dim=config['embedding_dim'],

        hidden_dims=config['hidden_dims'],

        dropout_rate=config['dropout_rate'],

        temperature=config['temperature']

    )

    model.compile(optimizer=tf.keras.optimizers.Adam(learning_rate=config['lr']))

    model.fit(train_ds.batch(config['batch_size']),

              validation_data=val_ds.batch(config['batch_size']),

              epochs=10,
    callbacks=[tune.integration.keras.TuneReportCallback({

        'val_recall': 'val_factorized_top_k/

top_100_categorical_accuracy'})])])


def run_search():

    search_space = {

        'embedding_dim': tune.choice([64, 128, 256]),

        'hidden_dims': tune.choice([[256, 128], [512, 256]]),

```

```
'dropout_rate': tune.uniform(0.1, 0.4),

'temperature': tune.loguniform(0.01, 0.1),

'lr': tune.loguniform(1e-4, 1e-2),

'batch_size': tune.choice([4096, 8192, 16384])

}

scheduler = ASHAScheduler(max_t=10, grace_period=2, reduction_factor=2)

search = OptunaSearch(metric='val_recall', mode='max')

return tune.run(train_with_config, config=search_space, num_samples=50,

                scheduler=scheduler, search_alg=search,

                resources_per_trial={'cpu': 4, 'gpu': 1})
```

Chapter 12: Model Evaluation & Offline Metrics

12.1 Retrieval Metrics

Retrieval Evaluation

```
import numpy as np

from collections import defaultdict


class RetrievalEvaluator:

    def __init__(self, ks=[10, 50, 100, 500, 1000]):

        self.ks = ks

    def recall_at_k(self, retrieved, relevant, k):

        retrieved_k = set(retrieved[:k])

        return len(retrieved_k & set(relevant)) / max(len(relevant), 1)

    def mrr(self, retrieved, relevant):

        relevant_set = set(relevant)

        for i, item in enumerate(retrieved):

            if item in relevant_set:

                return 1.0 / (i + 1)

        return 0.0

    def evaluate_batch(self, retrieved_lists, relevant_lists):

        metrics = defaultdict(list)
```

```

    for retrieved, relevant in zip(retrieved_lists, relevant_lists):

        for k in self.ks:

            metrics[f'recall@{k}'].append(self.recall_at_k(retrieved,
relevant, k))

            metrics['mrr'].append(self.mrr(retrieved, relevant))

    return {k: np.mean(v) for k, v in metrics.items()}

```

12.2 Ranking Metrics

Ranking Evaluation

```

class RankingEvaluator:

    def __init__(self, ks=[5, 10, 20]):

        self.ks = ks

    def dcg_at_k(self, relevances, k):

        relevances = np.asarray(relevances)[:k]

        gains = 2 ** relevances - 1

        discounts = np.log2(np.arange(len(relevances)) + 2)

        return np.sum(gains / discounts)

    def ndcg_at_k(self, relevances, k):

        dcg = self.dcg_at_k(relevances, k)

        ideal = self.dcg_at_k(np.sort(relevances)[::-1], k)

        return dcg / max(ideal, 1e-10)

```

```

def evaluate(self, predictions, labels):

    sorted_idx = np.argsort(predictions)[::-1]

    sorted_labels = labels[sorted_idx]

    metrics = {}

    for k in self.ks:

        metrics[f'ndcg@{k}'] = self.ndcg_at_k(sorted_labels, k)

    from sklearn.metrics import roc_auc_score

    metrics['auc'] = roc_auc_score(labels, predictions)

    return metrics

```

12.3 Diversity Metrics

Diversity Evaluation

```

def intra_list_diversity(embeddings, k=None):

    if k: embeddings = embeddings[:k]

    n = len(embeddings)

    if n < 2: return 0.0

    norm = embeddings / np.linalg.norm(embeddings, axis=1, keepdims=True)

    sim = norm @ norm.T

    dist = 1 - sim

    mask = ~np.eye(n, dtype=bool)

    return dist[mask].mean()

```

```

def category_coverage(rec_categories, all_categories):

```

```
return len(set(rec_categories)) / len(set(all_categories))
```

```
def creator_entropy(creator_ids):  
  
    from collections import Counter  
  
    counts = Counter(creator_ids)  
  
    total = len(creator_ids)  
  
    probs = [c / total for c in counts.values()]  
  
    return -sum(p * np.log(p + 1e-10) for p in probs)
```

Appendix C: Loss Functions Reference

C.1 Retrieval Loss Functions

Loss Function	Description	Use Case
Sampled Softmax	Multi-class classification with negative sampling	Standard two-tower
In-Batch Negatives	Use other batch items as negatives	Memory-efficient
InfoNCE	Contrastive loss with temperature scaling	Representation learning
Triplet Loss	Margin between positive and negative pairs	Fine-grained similarity

C.2 Ranking Loss Functions

Loss Function	Description	Use Case
Binary Cross-Entropy	Pointwise binary classification	Click prediction
BPR Loss	Pairwise ranking with log-sigmoid	Implicit feedback
Hinge Loss	Pairwise ranking with margin	Hard boundaries
ApproxNDCG	Differentiable approximation of NDCG	Direct metric optimization
LambdaRank	Gradient-weighted pairwise loss	Learning to rank

Appendix D: Hyperparameter Tuning Guide

D.1 Two-Tower Model Hyperparameters

Parameter	Range	Default	Notes
Embedding Dim	64 - 256	128	Larger = more expressive
Hidden Layers	1 - 4	2	[256, 128] typical
Dropout Rate	0.1 - 0.5	0.2	Higher for overfitting
Temperature	0.01 - 0.1	0.05	Lower = sharper
Batch Size	4K - 32K	8192	Larger = better negatives
Learning Rate	1e-4 - 1e-2	1e-3	Use warmup

D.2 Ranking Model Hyperparameters

Parameter	Range	Default	Notes
Cross Layers (DCN)	2 - 6	3	Higher order interactions
Deep Layers	2 - 5	3	[512, 256, 128]
Attention Heads	2 - 8	4	For transformers
Sequence Length	20 - 100	50	Recent history
L2 Regularization	1e-6 - 1e-4	1e-5	Prevent overfitting

D.3 Training Optimization Tips

1. Start with smaller models and scale up. A 64-dim two-tower provides a strong baseline.
2. Use mixed precision training (FP16) to double effective batch size.
3. Implement learning rate warmup for the first 1000-5000 steps.
4. Monitor both loss and retrieval metrics; loss can decrease while quality degrades.

5. For multi-task models, use uncertainty weighting or GradNorm for automatic balancing.
6. Evaluate on time-stratified splits to simulate real deployment conditions.

End of Volume 2

Volume 3 covers: Model Serving, A/B Testing, Monitoring, and MLOps

Volume 4 covers: Advanced Topics, System Design Interviews, and Case Studies